- ***Ansible:*** This open source automation tool enables you to automate applications and systems' deployment, configuration, and orchestration. It is designed to be simple, lightweight, and extensible.
- ***Testinfra:*** It is a framework that enables you to write unit tests for infrastructure. It provides a simple syntax for defining test cases, which can be used to verify the functionality and configuration of infrastructure components.

Now that we have a basic understanding of the immutable infrastructure concepts, let's dive in and understand how to deploy Apache web server on an Amazon Linux 2 machine by building a flawless golden AMI using Packer, configuring and deploying the web server with Ansible, and testing its functionality with Testinfra. Before we begin, make sure you have the following:

- Access to AWS console to launch EC2 instance.
- Packer, Ansible, and Testinfra packages on the local machine.
- Basic understanding of Ansible, Packer and Python.

Steps to building a golden AMI

Define Packer template: First, we'll create a Packer template that defines the AMI we want to bake in. Create a 'webserver.pk.hcl' file that installs 'httpd' package, configures a virtual host 'helloworld.example.com', and runs the web server on an Amazon Linux 2 AMI.

```
variable "app_role" { type = string }
variable "app_version" { type = string }
variable "instance_type" { type = string }
variable "root_volume_size" { type = string }
variable "aws_region" { type = string }
variable "aws_access_key" { default = "${env("aws_access_
key")}" }
variable "aws_secret_key" { default = "${env("aws_secret_
key")}" }

locals {
    timestamp = regex_replace(timestamp(), "[- TZ:]", "")
    ami_name  = lower("${var.app_role}-${var.app_version}")
}

source "amazon-ebs" "webserver" {
  region                  = "${var.aws_region}"
  ami_name                = "${local.ami_name}-${local.
timestamp}"
  ami_description         = "Golden AMI To Deploy WebServer"
  ssh_username            = "ec2-user"
  ssh_wait_timeout        = "10000s"
  instance_type           = "${var.instance_type}"
  source_ami              = "ami-052f483c20fa1351a"
  launch_block_device_mappings {
    device_name           = "/dev/sda1"
    volume_size           = "${var.root_volume_size}"
    volume_type           = "gp3"
    delete_on_termination = true
  }

  tags = {
    Name = "${var.app_role}"
  }
}

build {

  sources = ["source.amazon-ebs.webserver"]

  provisioner "file" {
    source          = "test_apache.py"
    destination     = "/tmp/"
  }

  provisioner "ansible" {
    playbook_file = "webserver.yaml"
  }

  provisioner "shell" {
    execute_command  = "{{.Vars}} sudo -E -S bash '{{.
Path}}'"
    inline = [
      "python3 -m pytest /tmp/test_apache.py",
    ]
  }
}
```

Define Ansible playbook: In this step, we will define an Ansible playbook that installs 'http' package, 'pytest-testinfra' module, configures virtual hosts, and finally starts the 'httpd' service on the Amazon Linux2 machine during the baking process.

```
- name: WebApp Playbook
  hosts: all
  become: true

  tasks:
    - name: install_httpd
      yum:
        name: httpd
        state: present

    - name: set_listener
      lineinfile:
```